

LAB EXPERIMENT – 5

Graph and Greedy Algorithms

PART 1 – Bellman-Ford Algorithm

Aim:

To implement the Bellman-Ford algorithm to find shortest paths from a single source vertex in a weighted graph and detect negative weight cycles.

Theory:

Bellman-Ford repeatedly relaxes all edges to compute shortest distances from a source. Unlike Dijkstra's algorithm, it can handle negative edge weights and can detect negative cycles.

Algorithm / Key Steps:

1. Initialize the source distance to 0 and all other distances to infinity.
2. Relax every edge $V - 1$ times, updating distance and parent when a shorter path is found.
3. Perform one additional relaxation pass; if any distance can still be reduced, a negative weight cycle exists.
4. Use the parent array to reconstruct and display the path from the source to each vertex.

Code:

```
class Solution {
public:
    vector<int> bellmanFord(int V, vector<vector<int>>& edges, int src) {

        vector<int> dis(V, 1e8);
        dis[src] = 0;
        for(int i = 0; i < (V - 1); i++) {
            for(int j = 0; j < edges.size(); j++) {
                int u = edges[j][0];
                int v = edges[j][1];
                int w = edges[j][2];
                if(dis[u] != 1e8 && dis[u] + w < dis[v]) {
                    dis[v] = dis[u] + w;
                }
            }
        }
        for(int i = 0; i < edges.size(); i++) {
            int u = edges[i][0];
            int v = edges[i][1];
            int w = edges[i][2];
            if(dis[u] != 1e8 && dis[u] + w < dis[v]) { // detecting the negative weight cycle
                return {-1};
            }
        }
        return dis;
    }
};
```

Output:

Input:  

V =

5

E =

5

edges[][] =

```
1 3 2
4 3 -1
2 4 1
1 2 1
0 1 5
```

src =

0

Your Output:

0 5 6 6 7

Expected Output:

0 5 6 6 7

PART 2 – Huffman Coding

Aim:

To implement Huffman Coding using a greedy approach and generate variable-length prefix codes for data compression.

Theory:

Huffman Coding repeatedly combines the two least-frequent nodes to build a binary tree. Characters with higher frequencies get shorter codes, reducing the total number of bits required.

Algorithm / Key Steps:

5. Read each character and its frequency from the user.
6. Insert all characters into a min-priority queue according to their frequencies.
7. Remove the two nodes with the smallest frequencies and combine them into a new node.
8. Repeat until only one node remains; this node is the root of the Huffman tree.
9. Traverse the tree, assigning 0 to a left edge and 1 to a right edge to generate Huffman codes.
10. Calculate original size and compressed size in bits using the given frequencies and generated codes.

Code:

```

class Solution {
public:
    struct Node {
        int freq;
        int idx;
        Node *left, *right;
        Node(int f, int i) : freq(f), idx(i), left(NULL), right(NULL) {}
    };

    struct Compare {
        bool operator()(Node* a, Node* b) {
            if (a->freq == b->freq) return a->idx > b->idx;
            return a->freq > b->freq;
        }
    };

    void preorder(Node* root, string code, vector<string> &ans) {
        if (!root) return;
        if (!root->left && !root->right) { // leaf node
            ans.push_back(code);
            return;
        }
        preorder(root->left, code + "0", ans);
        preorder(root->right, code + "1", ans);
    }

    vector<string> huffmanCodes(string &s, vector<int> f) {
        int n = s.size();
        priority_queue<Node*, vector<Node*>, Compare> pq;

        for (int i = 0; i < n; i++)
            pq.push(new Node(f[i], i));

        while (pq.size() > 1) {
            Node* left = pq.top(); pq.pop();
            Node* right = pq.top(); pq.pop();

            Node* parent = new Node(left->freq + right->freq,
                                    min(left->idx, right->idx));
            parent->left = left;
            parent->right = right;
            pq.push(parent);
        }

        vector<string> ans;

        if (n == 1) {
            ans.push_back("0");
            return ans;
        }

        preorder(pq.top(), "", ans);
        return ans;
    }
}

```

Output:

Input:  

s =

abcdef

f[] =

5 9 12 13 16 45

Your Output:

[0, 100, 101, 1100, 1101, 111]

Expected Output:

[0, 100, 101, 1100, 1101, 111]